

Introducción y Conceptos Básicos de Python

Estructura de Datos

2024-2025



Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

- 1 Entornos de Desarrollo Integrados
- 2 Revisando y Ampliando Código en Python
 - Trabajando con `list`
 - Leyendo Archivos
 - Usando F-Strings
 - Anotaciones de Tipo
 - Comentando Código
 - Definiendo una Función `main`
- 3 Creando y Usando una Clase

- 1 Entornos de Desarrollo Integrados
- 2 Revisando y Ampliando Código en Python
- 3 Creando y Usando una Clase

IDE

- ¿Notepad? → [Notepad++](#)
- Hay herramientas más potentes para programadores
 - [IDLE](#)
 - [Spyder](#)
 - [PyCharm](#)
- Otras opciones para Python
 - [Jupyter Notebooks](#) (Pruébalo [aquí](#))

Instalando Spyder

- Usando Anaconda
- Directamente desde la página web
- Usando [WinPython](#)
- Usando un gestor de paquetes
- Usando pip si Python está instalado de forma independiente
 - Sigue las instrucciones en la página web
 - Requiere crear un entorno

1 Entornos de Desarrollo Integrados

2 Revisando y Ampliando Código en Python

- Trabajando con `list`
- Leyendo Archivos
- Usando F-Strings
- Anotaciones de Tipo
- Comentando Código
- Definiendo una Función `main`

3 Creando y Usando una Clase

Desempaquetando una list

Podemos usar un `*` antes del nombre de una variable para desempaquetar una lista en sus elementos individuales

```
1 lst = ['uno', 'dos', 'tres', 'cuatro']
2
3 for element in lst:
4     print(element, end=' ')
5 # Es equivalente a
6 print(*lst)
7 # Estamos llamando a print con todos los
   elementos de la lista como parámetros
8 print(lst[0], lst[1], lst[2], lst[3])
```

- Podemos trabajar con los parámetros `end` y `sep` para modificar cómo se muestra la lista

Asignación Múltiple de Variables

```
1 a, b = 1, 2
2
3 def devuleve_dos():
4     # Hacer algo
5     return 'uno', 'dos'
6
7 primero, segundo = devuleve_dos()
```

- Python permite asignar más de una variable en una sola declaración
- Intenta hacer coincidir la estructura del lado izquierdo con la del lado derecho

Asignación de Variables con Desempaquetado

- Podemos usar el mismo truco con el operador de desempaquetado
- Solo una variable puede estar precedida por un asterisco

```
1 lst = ['uno', 'dos', 'tres', 'cuatro']  
2  
3 primero, segundo, *resto = lst  
4  
5 *inicio, ultimo = lst  
6  
7 primero, *mitad, ultimo = lst
```

Usando enumerate

- Al iterar a través de una lista, podríamos necesitar el índice o el elemento

```
1 lst = ['uno', 'dos', 'tres', 'cuatro']
2
3 for element in lst:
4     # Conocemos el elemento, pero no el índice
5
6 for i in range(len(lst)):
7     # Muy verboso, para acceder al elemento usamos lst[i]
```

Usando enumerate

- Al iterar a través de una lista, podríamos necesitar el índice o el elemento

```
1 lst = ['uno', 'dos', 'tres', 'cuatro']  
2  
3 for incide, valor in enumerate(lst):  
4     # Tenemos el índice y el elemento al  
    mismo tiempo
```

¿Por Qué Usar Archivos?

- Añadir persistencia a un programa
- Procesar la salida de otro programa
- Añadir opciones de configuración

Interactuando con Archivos

- La mayoría de las operaciones con archivos se realizan a través del módulo `os`¹
- La interacción más básica es comprobar si un archivo existe

```
1 import os.path
2 nombre_fichero = "mi_archivo.txt"
3 if os.path.isfile(nombre_fichero):
4     print("El archivo existe")
5 else
6     print("El archivo no existe")
```

- `os.path.exists(file_path)` también comprueba directorios

¹<https://docs.python.org/3/library/os.html>

Ayuda, No Puedo Acceder a mi Archivo

- Un archivo se localiza por su **ruta**
 - Ruta absoluta:
 - /home/ugaitz/Desktop/mi_archivo.txt
 - C:\Users\Ugaitz\Desktop\mi_archivo.txt
 - Ruta relativa:
 - Siempre relativa al directorio de trabajo actual (por defecto, la carpeta donde ejecutas el archivo de Python)
 - mi_archivo.txt
- Intenta usar rutas relativas
- Si no se encuentra un archivo, comprueba el directorio de trabajo actual del programa

```
1 import os
2 directorio_actual = os.getcwd()
```

Abriendo un Archivo

```
1 open(nombre_fichero, mode='r')
```

Modo	Descripción
r	Abrir archivo de texto para lectura
r+	Abrir el archivo para lectura y escritura
w	Abrir el archivo para escritura. Trunca el archivo si ya existe. Crea un nuevo archivo si no existe.
w+	Abrir el archivo para lectura y escritura. Trunca el archivo si ya existe. Crea un nuevo archivo si no existe.
a	Abrir el archivo para escritura. Los datos que se escriban se insertarán al final del archivo. Crea un nuevo archivo si no existe.
a+	Abrir el archivo para lectura y escritura. Los datos que se escriban se insertarán al final del archivo. Crea un nuevo archivo si no existe.

Abriendo un Archivo

Se recomienda comprobar si el archivo existe antes de abrirlo para leer

```
1 fichero = open("mi_archivo.txt", mode="r")
2 # Hacer cosas
3 fichero.close() # No olvides cerrar el archivo
```

```
1 with open("mi_archivo.txt", mode="r") as fichero:
2     # Hacer cosas
3 # Cuando salimos de la declaración with, el archivo
   se cierra automáticamente
```

Leer el Contenido

- `file.readline()` -> `str`
 - Leer una sola línea
- `file.read()` -> `str`
 - Leer todo el archivo y almacenarlo en string
- `file.readlines()` -> `lst[str]`
 - Leer todo el archivo y almacenarlo en una lista de strings
 - Combinación de `file.read()` y `str.splitlines()`

Iterar Sobre un Archivo

- Podemos iterar sobre cada línea de un archivo con una declaración **for**

```
1 with open("mi_archivo.txt", mode="r") as fichero:
2     for linea in fichero:
3         print(linea)
4         # podríamos tener que usar la función str.
           strip() para eliminar cualquier nueva l
           ínea al final
```

Formateando la Salida

```
1 print('Hola', nombre, '! Bienvenido a', asignatura,  
    '.')
```

```
2  
3 print('Hola %s! Bienvenido a %s.' % (nombre,  
    asignatura))
```

```
4  
5 print('Hola {}! Bienvenido a {}'.format(nombre,  
    asignatura))
```

```
6  
7 print(f'Hola {nombre}! Bienvenido a {asignatura}.')
```

Usando F-Strings

- Creamos una F-String añadiendo una f al principio de cualquier cadena de caracteres
- Dentro de las llaves podemos escribir lo que queramos
 - Tiene que ser código Python válido

```
1 print(f'Hola {nombre.upper()}! Bienvenido a  
    {getAsignatura(nombre)}.')
```

Formateando F-Strings

- Podemos pedirle a Python que formatee la salida de varias maneras
- Sigue la misma especificación que `str.format()`
- Usa dos puntos para separar el valor del formato

```
1 numero = 123.123141241241
2 # El punto especifica el número de decimales
3 # La f al final es para tratarlo como float
4 print(f'Número con 3 decimales={numero:.3f}')
```



```
5
6 # Podemos especificar la dirección con <, >, ^
7 # Podemos reservar un espacio de 10 de ancho para
   el número
8 print(f'Más ancho y a la derecha={numero:>10.3f}')
```

Formateando F-Strings, Uso Básico

Estructura básica del formato, todos los elementos son opcionales

[relleno] [alineación] [ancho] [.decimales] [tipo]

relleno: usado si hay espacio extra reservado, pero no utilizado, por defecto ' '

alineación: usado para especificar la alineación del texto

ancho: usado para reservar un ancho específico para el texto, por defecto el necesario para mostrar la variable

.decimales: usado para especificar cuántos decimales mostrar, el punto es obligatorio si lo usamos. Solo funciona con tipos numéricos

tipo: usado para forzar el tipo de la variable a mostrar

Tipado en Python

- Python no pide al programador especificar los tipos de las variables
 - Lo hace en tiempo de ejecución
- Podemos especificar el tipo de variables, parámetros de funciones o retornos
 - No es obligatorio
 - Útil como documentación
 - Python no lo tiene en cuenta al ejecutar el código
 - Introducido en la versión 3.5, mejorado en la 3.9

Tipado Simple

Podemos añadir un tipo a cualquier variable añadiendo dos puntos después del nombre

```
1 a: int = 5
2 b: float = 2.3
3 c: list = []
4
5 def get(lst: list, i: int, j: int) -> int:
6     # hacer algo
```

Complicando los Tipos

- Desde la versión 3.9 de Python podemos especificar el contenido de las colecciones
- Para leer más, ve [aquí](#)

```
1  # m es una lista de listas que contienen
    enteros
2  m: list[list[int]] = ...
3
4  # d es un diccionario con cadenas como
    claves y floats como valores
5  d: dict[str, float] = ...
6
7  # podemos hacerlos aún más complejos
8  a: dict[str, list[list[int]]] = ...
```

Cómo Comentar Código

- Usando `#` para comentarios de una sola línea
 - Sepáralo del código con dos espacios
 - Deja un espacio en blanco después de la almohadilla
- Usando `"""` para crear cadenas multilínea
 - No se asigna a una variable
 - A menudo se usa para documentación

```
1  # Esto es un comentario de una sola línea
2
3  """Esto también es un comentario"""
4
5  """
6  Esto es un comentario multilínea
7  """
```

Documentando Funciones

- Podemos usar comentarios para documentar funciones u otras piezas de código que hayamos creado
- La documentación ayuda a entender el código escrito por otra persona
- Hay varios estándares para escribir *docstrings*
 - Vamos a usar el de [NumPy](#)

La Variable `__name__`

- Cada vez que ejecutamos un archivo en Python se crean varias variables en segundo plano
- Una de ellas es la variable `__name__`. Esta variable tiene una de dos informaciones:

nombre del archivo: Si no estamos ejecutando el archivo directamente (lo estamos importando), su variable `__name__` se define como el nombre del archivo

`__main__`: Si estamos ejecutando el archivo directamente, entonces su `__name__` contendrá la cadena `'__main__'`

Definiendo el Script Ejecutado

- Usando la variable `__name__` podemos diferenciar entre lanzar el script directamente o importarlo
- Podemos tener piezas de código que se ejecutan solo si el script no está siendo importado
 - Por ejemplo, con fines de prueba

```
1 def main():  
2     # Esta función es el punto de entrada al  
3     programa  
4 # Usaremos este código al final del archivo  
5 if __name__ == "__main__":  
6     main()
```

- 1 Entornos de Desarrollo Integrados
- 2 Revisando y Ampliando Código en Python
- 3 Creando y Usando una Clase

¿Qué es una Clase?

- La mayoría de los elementos internos de Python están definidos usando clases
 - `list`, `dict`, `set`, `int`, `float` y más
- Una clase es una plantilla que define un comportamiento para un elemento con un equivalente en la “vida real”
- Cuando programamos estamos creando instancias de clases
 - Estamos trabajando con **objetos**

```
1 # Creamos un objeto de la clase list
2 lst = list()
```

Partes de una Clase

- Una clase se divide en dos componentes principales

Atributos: Variables que están dentro de la clase, el acceso directo a los atributos de una clase a menudo no está permitido

Métodos: Funciones que están definidas dentro de la clase, pueden usar los atributos en su ejecución

```
1 lst = list()
2 # Llamamos al método append() de la clase
   list
3 lst.append(5)
4
5 # Sabemos que lst contiene el elemento en
   algún atributo, pero no sabemos cómo
```

Creando una Clase

- Creamos una clase usando la palabra clave `class`
- El método `__init__` es una función especial que se llama cada vez que creamos un objeto de esa clase
 - Cada vez que hacemos `var = MiClase()`
- El método `__init__` también se usa para crear los atributos de una clase
 - `self` se refiere al objeto que se está usando
 - Los guiones bajos en los nombres de los atributos indican a cualquier otro programador que no los acceda directamente

```
1 class MiClase:
2     def __init__(self, param1, param2):
3         self.__attr1 = param1
4         self.__attr2 = param2
```

¿Por Qué el ____?

- Python no ofrece una forma de controlar quién puede acceder a los atributos, a diferencia de la mayoría de los otros lenguajes de programación orientados a objetos
 - `public`
 - Cualquiera puede acceder y modificar el atributo
 - `protected`
 - El nombre del atributo comienza con un guion bajo
 - El atributo se usa internamente por la clase, pero el acceso es posible
 - `private`
 - El nombre del atributo comienza con dos guiones bajos
 - El atributo se usa internamente por la clase, se hacen algunas modificaciones a su nombre
 - Usaremos esto para nuestros atributos

Añadiendo un Método

- Todos los métodos tendrán *self* como su primer parámetro
- Pueden tener tantos parámetros como sea necesario
- Pueden llamar a cualquier otra función o método

```
1 class MyClass:
2     def __init__(self, param1, param2):
3         self.__attr1 = param1
4         self.__attr2 = param2
5     def myMethod1(self, param):
6         # Cálculos complicados
7         self.__attr1 = param
8     def myMethod2(self, param):
9         self.myMethod1(param)
10        self.__attr2 = param
```

Usando un Método

- Cada vez que llamamos a un método, Python maneja automáticamente la definición del parámetro self
- Si lo hacemos manualmente, no estamos usando la clase correctamente

```
1 lst = list()
2 # Llamar al método
3 lst.append(5)
4 # Llamada incorrecta al método
5 list.append(lst, 5)
6
7 var = MyClass(5, 6)
8 var.myMethod2(8)
```